

Partie 1 : Planificateur de repas — Patron de conception Monteur (Builder)

Description :

Le planificateur de repas est une petite application qui permet aux utilisateurs de créer des repas personnalisés en utilisant le patron de conception Monteur (Builder). L'application fournit une interface où les utilisateurs peuvent sélectionner divers aliments, spécifier des préférences alimentaires et générer des plans de repas personnalisés.

Classes pour l'application :

- **MealPlanDirector** : La classe Directeur (Director) qui orchestre le processus de construction et interagit avec les monteurs.
- **MealPlanBuilder** : La classe abstraite Monteur (Builder) qui définit les méthodes de construction communes pour créer différents composants du plan de repas.
- **BreakfastBuilder, LunchBuilder, DinnerBuilder, SnackBuilder** : Classes Monteurs concrètes (Concrete Builders) qui implémentent les méthodes de construction spécifiques à chaque composant du plan de repas.
- **MealPlan** : La classe Produit (Product) qui représente le plan de repas final généré, contenant des informations sur les aliments sélectionnés, les préférences alimentaires, les portions et l'analyse nutritionnelle.

Partie 2 : Générateur de boutique en ligne — Patron de conception Monteur (Builder)

Description :

Le générateur de boutique en ligne est une mini-application qui permet aux utilisateurs de créer leurs propres boutiques en ligne personnalisées en utilisant le patron de conception Monteur (Builder). L'application offre une approche flexible et modulaire pour construire des boutiques en ligne en séparant le processus de construction de la représentation de la boutique finale.

Classes pour l'application :

- **StoreBuilder (Monteur abstrait)** : Fournit des méthodes pour construire différents composants de la boutique, tels que la disposition (layout), le catalogue de produits, le panier d'achats et le système de paiement. Inclut une méthode abstraite pour récupérer la boutique construite.
- **OnlineStore (Produit)** : Contient des attributs pour stocker les composants construits, tels que la disposition, le catalogue de produits, le panier d'achats et le système de paiement. Fournit des méthodes d'accès (getters) et de modification (setters) pour accéder aux composants de la boutique et les modifier.
- **BasicStoreBuilder (Monteur concret)** : Implémente les méthodes pour construire la disposition, le catalogue de produits, le panier d'achats et le système de paiement selon des spécifications prédéfinies. Retourne la boutique en ligne construite.

- **AdvancedStoreBuilder (Monteur concret)** : Implémente les méthodes pour construire la disposition, le catalogue de produits, le panier d'achats et le système de paiement avec des fonctionnalités supplémentaires et des options de personnalisation. Retourne la boutique en ligne construite.
- **StoreDirector (Directeur)** : Contrôle le processus de construction de la boutique en ligne. Prend un objet StoreBuilder en entrée et l'utilise pour construire la boutique. Fournit une méthode pour initier le processus de construction et récupérer la boutique construite.

Partie 3 : Système de commande de pizza en ligne — Patron de conception Fabrique (Factory)

Description du projet :

Le système de commande de pizza en ligne permet aux clients de commander des pizzas via une application web. Le système propose différents types de pizzas, telles que Margherita, Pepperoni et Végétarienne. Chaque type de pizza possède son propre ensemble d'ingrédients et son propre processus de préparation.

Classes pour l'application :

- **Pizza (Produit)** : Représente la classe de base abstraite pour les différents types de pizzas. Contient des méthodes pour préparer et cuire la pizza.
- **MargheritaPizza, PepperoniPizza, VegetarianPizza (Produits concrets)** : Héritent de la classe Pizza et représentent différents types de pizzas. Implémentent les méthodes de préparation et de cuisson spécifiques selon leur type de pizza.
- **PizzaFactory (Créateur)** : Contient une méthode statique pour créer des pizzas en fonction de la commande du client. Encapsule la logique de création et masque les détails d'instanciation au client. Retourne l'objet pizza concret approprié selon le type demandé.
- **OnlinePizzaOrderingSystem** : Représente le code client qui interagit avec la PizzaFactory pour commander des pizzas. Prend les entrées du client concernant le type de pizza souhaité. Utilise la PizzaFactory pour créer l'objet pizza demandé. Interagit avec l'objet pizza créé pour finaliser le processus de commande.

Partie 4 : Système de fabrication de véhicules — Patron de conception Fabrique (Factory)

Description du projet :

Le système de fabrication de véhicules est responsable de la production de différents types de véhicules, tels que des voitures, des motos et des vélos. Chaque type de véhicule a ses propres spécifications uniques et son propre processus de fabrication. Le système vise à fournir une solution flexible et extensible pour produire des véhicules efficacement.

Classes et implémentation :

- **Vehicle (Produit)** : Représente la classe de base abstraite pour les différents types de véhicules. Contient des méthodes et des attributs communs à tous les véhicules, tels que le modèle, la couleur et le prix.
- **Car, Motorcycle, Bicycle (Produits concrets)** : Héritent de la classe Vehicle et représentent des types spécifiques de véhicules. Implémentent les méthodes et attributs spécifiques selon leur type de véhicule.
- **VehicleFactory (Créateur)** : Contient une méthode statique pour créer des véhicules en fonction de la demande du client. Encapsule la logique de création et masque les détails d'instanciation au client. Retourne l'objet véhicule concret approprié selon le type demandé.
- **VehicleManufacturingSystem** : Représente le code client qui interagit avec la VehicleFactory pour fabriquer des véhicules. Prend les entrées de l'utilisateur concernant le type de véhicule à fabriquer. Utilise la VehicleFactory pour créer l'objet véhicule demandé. Interagit avec l'objet véhicule créé pour exécuter les tâches de fabrication.

Livrables

1. Créez un fichier ZIP contenant les fichiers de code source nécessaires (Python). Vous devez nommer votre fichier selon la convention suivante : **A#_IDétudiant**, où **#** est le numéro du devoir et **IDétudiant** est votre numéro d'identifiant étudiant.
2. Les devoirs doivent être remis dans la section des devoirs (Léa) au plus tard le **26 juillet 2026 à 23h55**.